

Lab - 2:

Experiment No: 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing

- **Flask** → to run the blockchain API server
-

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
 - store the chain
 - implement Proof of Work
 - validate chain integrity
-

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
 - `/get_chain` → fetch full blockchain
 - `/is_valid` → verify blockchain
-

CODE

```
!pip install flask
```

```
Requirement already satisfied: flask in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: blinker>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in /usr/local/lib/python3.12/dist-packages (from flask) (8.4.2)
Requirement already satisfied: itsdangerous>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from flask) (3.1.8)
```

```
import datetime
import hashlib
import json
from flask import Flask, jsonify, request
```

```

class Blockchain:

    def __init__(self):
        # Store all blocks
        self.chain = []

        # Store pending transactions
        self.transactions = []

        # Create Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0',
            transactions=[]
        )

# -----
# CREATE TRANSACTION
# -----

    def create_transaction(self, sender, receiver, amount):

        transaction = {
            'sender': sender,
            'receiver': receiver,
            'amount': amount
        }

        self.transactions.append(transaction)

        return len(self.chain) + 1

# -----
# CREATE BLOCK
# -----

    def create_block(self, proof, previous_hash, transactions):

        block = {
            'index': len(self.chain) + 1,

```

```

        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': transactions
    }

    self.chain.append(block)

    return block

# -----
# GET PREVIOUS BLOCK
# -----

def get_previous_block(self):

    return self.chain[-1]

# -----
# PROOF OF WORK
# -----

def proof_of_work(self, previous_proof):

    new_proof = 1

    check_proof = False

    while check_proof is False:

        hash_operation = hashlib.sha256(
            str(new_proof ** 2 - previous_proof ** 2).encode()
        ).hexdigest()

        # Cryptographic puzzle:
        # Hash must start with 000

        if hash_operation[:3] == '000':
            check_proof = True

```

```

        else:
            new_proof += 1

    return new_proof

# -----
# HASH BLOCK
# -----

def hash(self, block):

    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    return hashlib.sha256(encoded_block).hexdigest()

# -----
# CHECK BLOCKCHAIN VALIDITY
# -----

def is_chain_valid(self, chain):

    previous_block = chain[0]

    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        # Check previous hash

        if block['previous_hash'] != self.hash(previous_block):

            return False

        # Check Proof of Work

```

```

previous_proof = previous_block['proof']

proof = block['proof']

hash_operation = hashlib.sha256(
    str(proof ** 2 - previous_proof ** 2).encode()
).hexdigest()

if hash_operation[:3] != '000':

    return False

previous_block = block

block_index += 1

return True

blockchain.create_transaction(
    sender="Bob",
    receiver="Charlie",
    amount=50
)

blockchain.create_transaction(
    sender="Charlie",
    receiver="David",
    amount=25
)

print(json.dumps(blockchain.transactions, indent=4))

```

```

... [
    {
        "sender": "Alice",
        "receiver": "Bob",
        "amount": 100
    },
    {
        "sender": "Bob",
        "receiver": "Charlie",
        "amount": 50
    },
    {
        "sender": "Charlie",
        "receiver": "David",
        "amount": 25
    }
]

```

```
block = mine_block(blockchain)
```

```
print(json.dumps(block, indent=4))
```

```

{
  "index": 2,
  "timestamp": "2026-08-19 19:14:59.280678",
  "proof": 533,
  "previous_hash": "fbdbe234da2ddb2b2df9e13ae9511ab4f43685130f7aee72998a999159b8fbff",
  "transactions": [
    {
      "sender": "Alice",
      "receiver": "Bob",
      "amount": 100
    },
    {
      "sender": "Bob",
      "receiver": "Charlie",
      "amount": 50
    },
    {
      "sender": "Charlie",
      "receiver": "David",
      "amount": 25
    }
  ]
}

```

```
print("Pending Transactions:")
```

```
print(blockchain.transactions)
```

... Pending Transactions: []

```
print(json.dumps(blockchain.chain, indent=4))

[
  {
    "index": 1,
    "timestamp": "2026-08-19 19:13:14.487238",
    "proof": 1,
    "previous_hash": "0",
    "transactions": []
  },
  {
    "index": 2,
    "timestamp": "2026-08-19 19:14:59.280678",
    "proof": 533,
    "previous_hash": "fbdbe234da2ddb2b2df9e13ae9511ab4f43685130f7aee72998a999159b8fbff",
    "transactions": [
      {
        "sender": "Alice",
        "receiver": "Bob",
        "amount": 100
      },
      {
        "sender": "Bob",
        "receiver": "Charlie",
        "amount": 50
      },
      {
        "sender": "Charlie",
        "receiver": "David",
        "amount": 25
      }
    ]
  }
]
```

```
result = blockchain.is_chain_valid(blockchain.chain)
```

```
print("Is Blockchain Valid?", result)
```

... Is Blockchain Valid? True

```
response = requests.get(
    'http://127.0.0.1:5000/get_chain'
)
```

```
print(response.status_code)
print(json.dumps(response.json(), indent=4))
```

```
INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 19:29:21] "GET /get_chain HTTP/1.1" 200 - 200
```

```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-08-19 19:20:46.573842",
      "transactions": []
    }
  ],
  "length": 1
}
```

```
response = requests.get(
    'http://127.0.0.1:5000/mine_block'
)
```

```
print(response.status_code)
```

```
print(
    json.dumps(
        response.json(),
        indent=4
    )
)
```

```
INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 19:30:12] "GET /mine_block HTTP/1.1" 200 - 200
```

```
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "47dad6cbe9fb4fca3615f6356e35a544946fda3dfeaf3c0fb3a2fb3dd4484b69",
  "proof": 533,
  "timestamp": "2026-08-19 19:30:12.520539",
  "transactions": [
    {
      "amount": 500,
      "receiver": "Bob",
      "sender": "Alice"
    }
  ]
}
```

```
response = requests.get(
    'http://127.0.0.1:5000/is_valid'
)
```

```
print(response.json())
```

```
INFO:werkzeug:127.0.0.1 - - [19/Aug/2026 19:31:18] "GET /is_valid HTTP/1.1" 200 -  
{'message': 'All good. The Blockchain is valid.'}
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.